Programming Assignment 1
THREE GENIUSES

CS683: Advanced Computer Architecture, Autumn 2025
Computer Science and Engineering

Task 1

Matrix Multiplication — The Rancho Way

Task 1A: Unroll Baba Unroll

Analyze the provided matrix multiplication code and implement loop reordering and unrolling optimizations to improve execution time.

By the end of this task, you should be able to answer the following questions:

1. What motivated you to make changes to your code?
    - **Cache use:** The original version (`ijk`) goes down columns of matrix B, which means the CPU keeps missing the cache and has to fetch from slower memory.

By reordering loops (`i-k-j`), we instead move through rows of B and C, which are stored next to each other in memory. This makes memory access faster.

- **Reusing values:** Each number from A (`A[i,k]`) is used many times. If we keep it in a register (fast CPU storage), we don't need to load it again and again.

- **Loop unrolling:** By handling multiple elements in one loop step, we reduce the "loop overhead" (the cost of checking conditions and jumping). This also lets the CPU work on several operations at once.

2. What considerations did you take into account when implementing those changes?
    - We wanted to access memory in long continuous chunks (rows), because that's how data is stored in RAM.
3. How effective are your changes? (Which metric will you use to quantify effectiveness and why?)
    - **Execution time (ms):** This tells us how long the program actually takes, which is the most direct way to measure speed.
    - **Speedup:** We calculate `baseline time ÷ optimized time`. This makes it easy to see how much faster the new version is.

Task 1B: Divide Karo, Rule Karo

Deliverables

1. ***Profiling baseline matrix multiplication code:***

- Find out the size of the **L1 Data (L1-D) cache** on your system.
    - 32KB (for each CPU core, shared between two logical cores)
    - 8-way set associative, 64 sets, Line size = 64B

- Profile the baseline (naive) matrix multiplication implementation and report the **L1-D cache misses per kilo instructions (MPKI)**. *Hint: Use a tool called **perf**.*
  - $MPKI = \frac{L1-dcache-load-misses}{L1-dcache-loads} \times 1000$
  - $MPKI = \frac{134,942,569}{2,979,174,885} \times 1000 = 45.29$

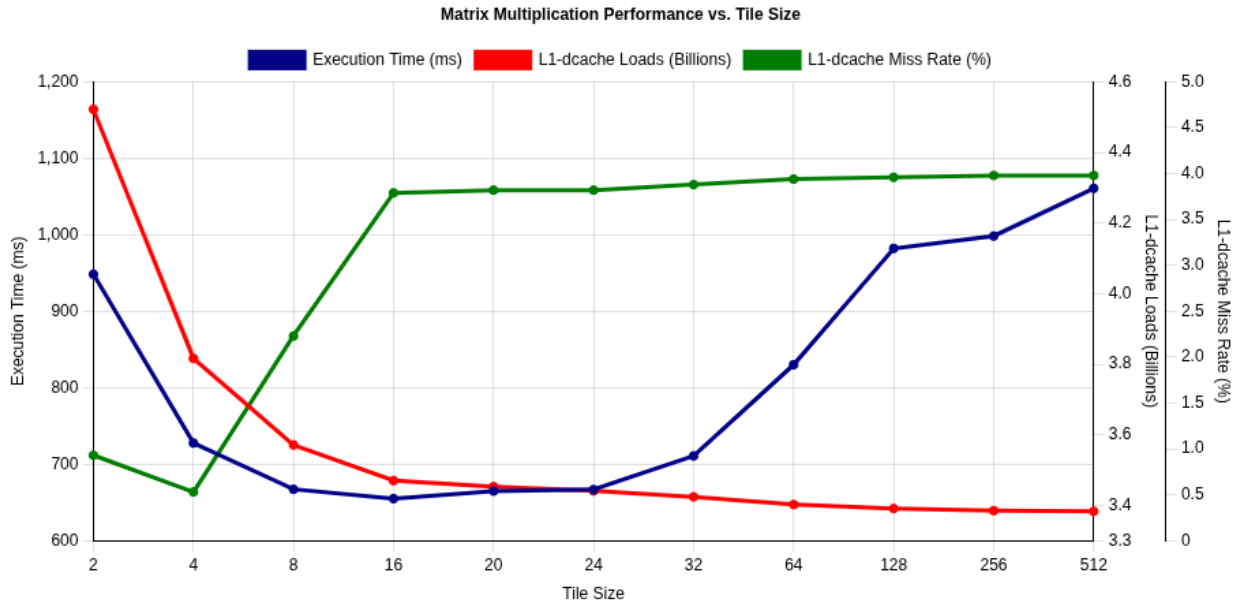
```
joy@joy:~/Documents/Advanced Computer Architure/PA-1-main/PA1/Task1$ perf stat -e L1-dcache-loads,
L1-dcache-load-misses taskset -c 0 ./mat_mul_nive 512
Normal matrix multiplication took 1147 ms to execute


 Performance counter stats for 'taskset -c 0 ./mat_mul_nive 512':

     2,979,174,885      L1-dcache-loads
       134,942,569      L1-dcache-load-misses           #    4.53% of all L1-dcache accesses

       1.167103857 seconds time elapsed

       1.157973000 seconds user
       0.007992000 seconds sys


joy@joy:~/Documents/Advanced Computer Architure/PA-1-main/PA1/Task1$ ^C
```

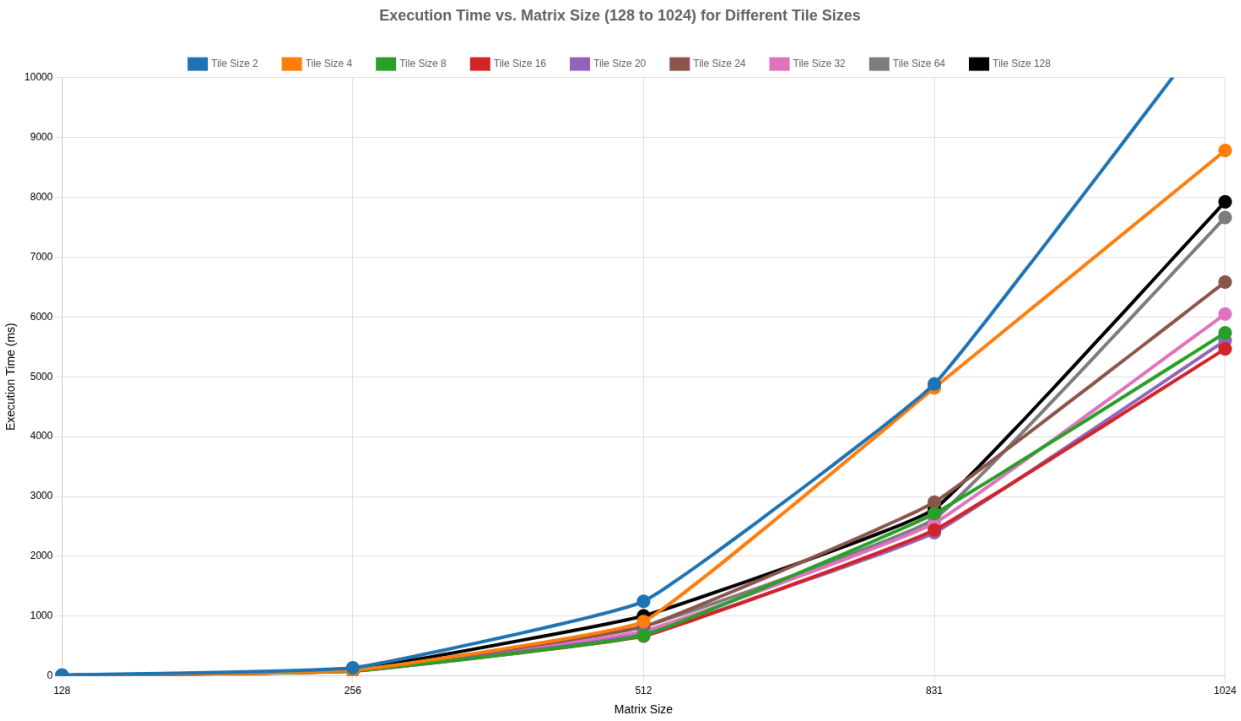
2. *Implementing Tiled matrix multiplication*

- Implement a **tiled** version of matrix multiplication.
- Experiment with different **tile sizes**.
- For each tile size, measure the **L1-D cache MPKI** to evaluate cache behavior.
- $MPKI = \frac{L1-dcache-load-misses}{L1-dcache-loads} \times 1000$
- All these tests are performed on a 512x512 matrix size.

Tile Size	L1-dcache-loads	L1-dcache-load-misses	Execution Time (ms)	MPKI
2	4,521,707,194	42,133,456	948.4	9.32
4	3,816,431,311	20,399,632	727.6	5.35

8	3,571,044,265	79,561,468	667.1	22.28
16	3,469,574,995	131,450,705	654.9	37.89
20	3,452,526,835	131,827,459	664.8	38.18
24	3,441,097,934	131,452,348	666.7	38.20
32	3,423,651,990	132,782,991	710.8	38.78
64	3,401,670,845	133,866,447	830.1	39.35
128	3,391,299,963	134,416,861	982.2	39.64
256	3,385,451,044	134,699,675	998.4	39.79
512	3,383,495,110	134,832,011	1060.8	39.85





- Identify the **tile size** that gives the best performance on your hardware in terms of cache efficiency and execution time.
- The best performance in terms of execution is achieved on a tile size of 16.

3. *Collecting and analyzing different metrics of interest:*

- Test your tiled implementation on matrices of various sizes.
- Calculate the **speedup** achieved by the tiled version compared to the baseline implementation for each matrix size.
- Create plots to visualize the performance trends:
    - **L1-D MPKI vs. Matrix Size** for different tile sizes.

Tile Size 2:

Matrix Size	L1-dcache-loads	L1-dcache-load-misses	Execution Time (ms)	MPKI

2	1,640,957	78,714	0	47.96835018
4	1,642,149	82,035	0	49.95588098
8	1,661,238	79,643	0	47.94195654
16	1,798,237	77,444	0	43.06662581
32	2,830,379	78,773	0	27.83125511
64	10,787,162	105,499	1.3	9.780051509
128	73,369,044	611,017	13	8.327994569
256	569,576,245	5,483,409	126.7	9.627172917
512	4,522,998,465	48,237,029	1126.9	10.66483426
1024	36,078,155,410	387,494,899	10891.5	10.7404299
2048	288,343,669,407	4,778,056,828	135446.5	16.57070134

Tile Size 4:

Matrix Size	L1-dcache-loads	L1-dcache-load-misses	Execution Time (ms)	MPKI
2	1,651,250	78,671	0	47.64330053
4	1,641,341	76,859	0	46.8269543
8	1,657,437	75,661	0	45.64939723
16	1,780,585	78,492	0	44.08214154
32	2,651,065	77,184	0	29.11433707
64	9,403,153	90,576	1	9.63251369
128	62,330,334	296,752	10.7	4.760956359
256	481,444,394	1,794,593	98.3	3.727518738
512	3,816,996,779	20,582,677	764.3	5.392374736
1024	30,433,507,415	172,657,418	7071.4	5.673267154
2048	243,085,845,661	1,796,271,876	63265.4	7.389454829

Tile Size 8:

Matrix Size	L1-dcache-loads	L1-dcache-load-misses	Execution Time (ms)	MPKI

2	1,639,037	80,059	0	48.84514505
4	1,645,699	80,359	0	48.82970701
8	1,657,861	80,041	0	48.27968087
16	1,769,460	78,427	0	44.32256169
32	2,595,722	77,413	0	29.82330157
64	8,923,067	84,746	1	9.497407113
128	58,489,475	167,634	9.4	2.866054106
256	450,651,189	1,040,488	76.1	2.308854443
512	3,571,025,291	81,619,977	654.2	22.85617445
1024	28,467,906,841	677,947,637	5442.3	23.81445326
2048	227,367,638,664	5,493,010,075	46050.8	24.15915522

Tile Size 16:

Matrix Size	L1-dcache-loads	L1-dcache-load-misses	Execution Time (ms)	MPKI
2	1,643,485	80,346	0	48.88757731
4	1,642,524	80,087	0	48.75849607
8	1,662,070	77,850	0	46.83918247
16	1,765,772	75,899	0	42.98346559
32	2,570,276	78,103	0	30.38700902
64	8,723,880	82,258	1	9.429061381
128	56,900,604	148,239	9.3	2.605227178
256	437,972,199	6,916,357	84.3	15.79177175
512	3,469,658,548	130,327,150	649.6	37.56195262
1024	27,655,059,325	997,286,424	5231.6	36.06162664
2048	220,861,925,021	8,029,532,877	43232.1	36.35544187

Tile Size 20:

Matrix Size	L1-dcache-loads	L1-dcache-load-misses	Execution Time (ms)	MPKI

2	1,640,624	82,514	0	50.29427827
4	1,647,247	77,904	0	47.29345387
8	1,656,059	76,280	0	46.06116086
16	1,764,964	77,641	0	43.99013238
32	2,573,499	79,459	0	30.87586201
64	8,734,554	87,435	1	10.01024208
128	56,761,836	174,039	9.3	3.066127036
256	435,915,199	15,141,527	81	34.73502882
512	3,452,535,026	131,511,139	675.5	38.0911817
1024	27,516,214,278	1,021,401,718	5282.4	37.11999433
2048	219,715,977,665	7,968,054,001	46683.8	36.26524609

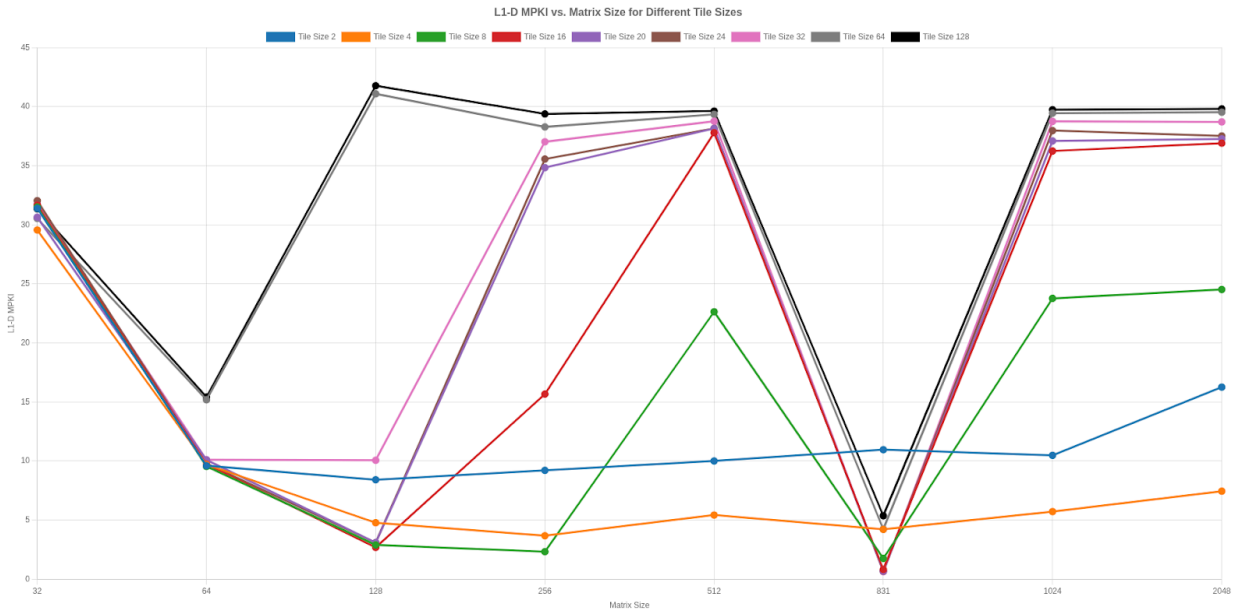
Tile Size 24:

Matrix Size	L1-dcache-loads	L1-dcache-load-misses	Execution Time (ms)	MPKI
2	1,641,547	80,521	0	49.05190043
4	1,647,468	78,837	0	47.85343327
8	1,657,299	78,996	0	47.66550876
16	1,771,903	77,641	0	43.81786136
32	2,571,185	79,482	0	30.91259478
64	8,685,052	85,306	1	9.822163414
128	56,579,654	179,668	9.5	3.175487782
256	434,479,238	15,291,244	80.3	35.1944182
512	3,440,950,978	131,715,342	654.7	38.27876155
1024	27,412,675,998	1,040,330,455	5363	37.95070773
2048	218,915,509,819	8,135,182,911	46583.2	37.16129075

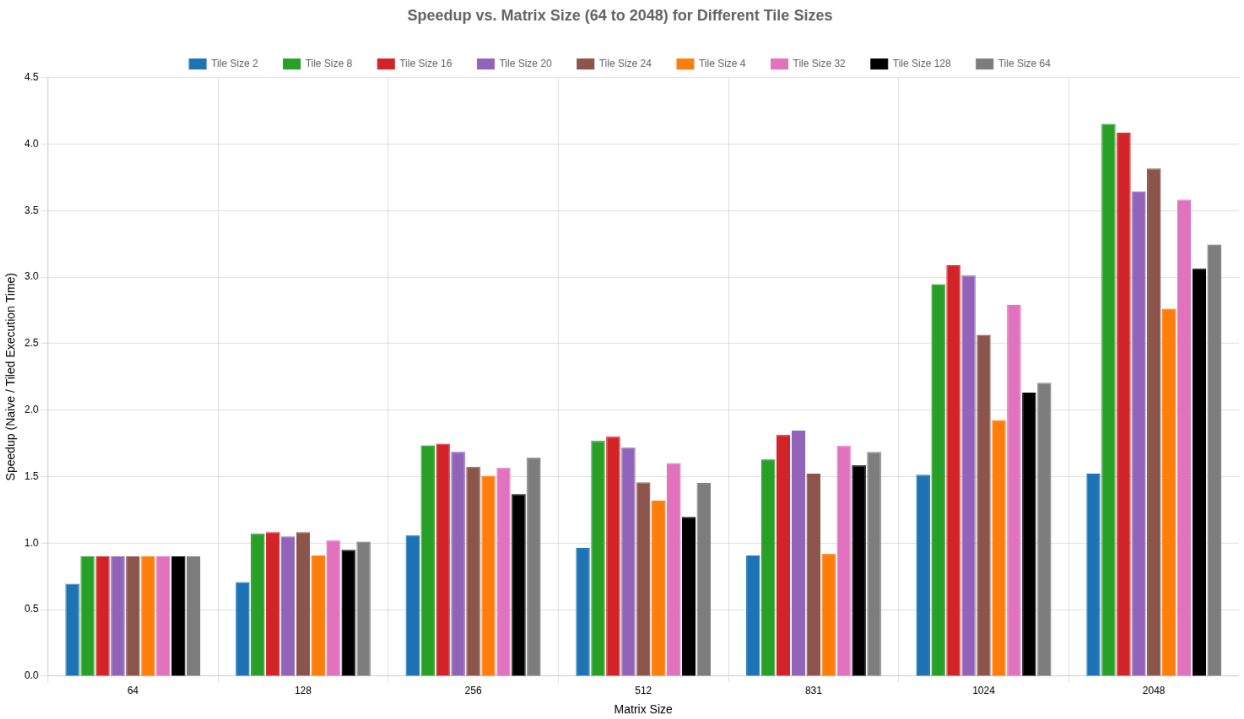
Tile Size 32:

Matrix Size	L1-dcache-loads	L1-dcache-load-misses	Execution Time (ms)	MPKI

32	2,554,353	81,101	0	31.75011441
64	8,629,179	87,239	1	10.10976826
128	56,182,597	565,314	10.5	10.06208382
256	432,287,865	16,008,855	92.8	37.03285772
512	3,423,784,968	132,747,338	751.3	38.77210141
831	14,596,793,395	9,231,570	2554.9	0.6324382178
1024	27,288,110,441	1,057,682,417	6049.1	38.75982616
2048	217,969,967,801	8,439,046,967	56567.3	38.7165583



- **Speedup vs. Matrix Size** for different tile sizes.



Answer the following questions:

1. Report the changes in **L1-D MPKI** observed when moving from naive to tiled matrix multiplication. Justify your observations.
    - For medium sizes (notably 128 and 256, also 831 here), tiling reduced L1-D MPKI a lot. For some sizes (32, 64, 512, 1024) MPKI actually rose with tile=16.
2. How did **L1-D MPKI** vary across different matrix sizes and tile sizes? Explain your findings in terms of the cache hierarchy and working set sizes.

Cache capacity (32KB L1):

- When tiles are small (like 4 or 8), the pieces of A, B, and C being multiplied fit inside the fast L1 cache, so the data gets reused efficiently.
- When tiles are large (64 or 128), they don't fit in L1, so the cache keeps throwing data out and bringing it back in, which causes many misses.

Spatial locality:

- Small or medium tiles process rows and columns in order, so memory is accessed in a smooth, continuous way. This makes better use of each cache line.

Matrix size vs. cache levels:

- For small matrices, the whole thing fits in cache already, so tiling doesn't help much.
- For medium-sized matrices, tiling helps a lot because only the active block stays in cache at a time.
- For very large matrices, tiling still helps, but now performance is also limited by slower, bigger caches (L2/L3) and memory bandwidth.

3. Did you achieve a **speedup**? If yes, quantify the improvement and identify the contributing factors. If not, analyze the limiting factors and propose possible solutions.

Yes, we achieved speedup

Much fewer L1 cache misses: Tiling keeps the working block small enough to stay in L1 while we compute it. That avoids repeated loads from slower memory and reduces stalls.

Better data reuse: Each element loaded from memory is used many times before being evicted. More useful work per load → faster overall.
Reduced memory traffic: Because data is reused in cache, the program touches DRAM far less often. For large N, DRAM accesses are the expensive part; reducing them yields big gains.
Improved write locality: Writes to C become more contiguous and cache-friendly, improving store buffering and reducing write-allocate penalties.

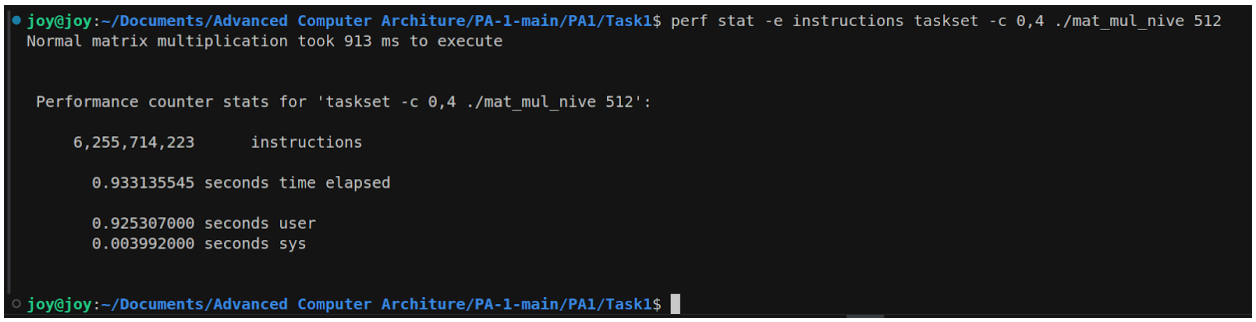
Task 1C: Data Ko Line Mein Lagao

In this task, you will utilize SIMD (Single Instruction, Multiple Data) instructions to parallelize matrix multiplication and analyze their impact on instruction count and overall performance.

Deliverables

1. ***Baseline Profiling:***

   - Report the **number of instructions** executed for the **naive** matrix multiplication implementation.
   - Number of instructions = 6,255,714,223 (Matrix size 512x512)



2. ***SIMD Implementation:***

   - Modify your matrix multiplication to leverage **SIMD instructions**.
   - Implement versions using **128-bit**, **256-bit**, and (if available) **512-bit** SIMD registers.

3. *Instruction Count and Performance Analysis:*

- Report the **number of instructions** executed when running only the **SIMD version**.

```
//128-bit SSE version (processes 2 doubles at a time)
```

- Number of instructions = 529,268,882 (Matrix size 512x512)

```
joy@joy:~/Documents/Advanced Computer Architure/PA-1-main/PA1/Task1$ perf stat -e instructions taskset -c 0,4 ./mat_mul/simd 512
SIMD matrix multiplication took 230 ms to execute

 Performance counter stats for 'taskset -c 0,4 ./mat_mul/simd 512':

       529,286,882      instructions

       0.248235004 seconds time elapsed

       0.241552000 seconds user
       0.004990000 seconds sys


joy@joy:~/Documents/Advanced Computer Architure/PA-1-main/PA1/Task1$
```

```
//256-bit AVX version (processes 4 doubles at a time)
```

- Number of instructions = 396,559,330 (Matrix size 512x512)

```
joy@joy:~/Documents/Advanced Computer Architure/PA-1-main/PA1/Task1$ perf stat -e instructions taskset -c 0,4 ./mat_mul/simd 512
SIMD matrix multiplication took 210 ms to execute

 Performance counter stats for 'taskset -c 0,4 ./mat_mul/simd 512':

       396,559,330      instructions

       0.227325506 seconds time elapsed

       0.223187000 seconds user
       0.002001000 seconds sys


joy@joy:~/Documents/Advanced Computer Architure/PA-1-main/PA1/Task1$
```

- **Compare performance** between the naive and SIMD implementations by calculating the **speedup** achieved.
- $Speedup(128 - bit) = \frac{Execution\ Time\ without\ SIMD}{Execution\ Time\ with\ SIMD}$

  $Speedup(128 - bit) = \frac{913\ ms}{230\ ms} = 3.97$
- $Speedup(256 - bit) = \frac{Execution\ Time\ without\ SIMD}{Execution\ Time\ with\ SIMD}$

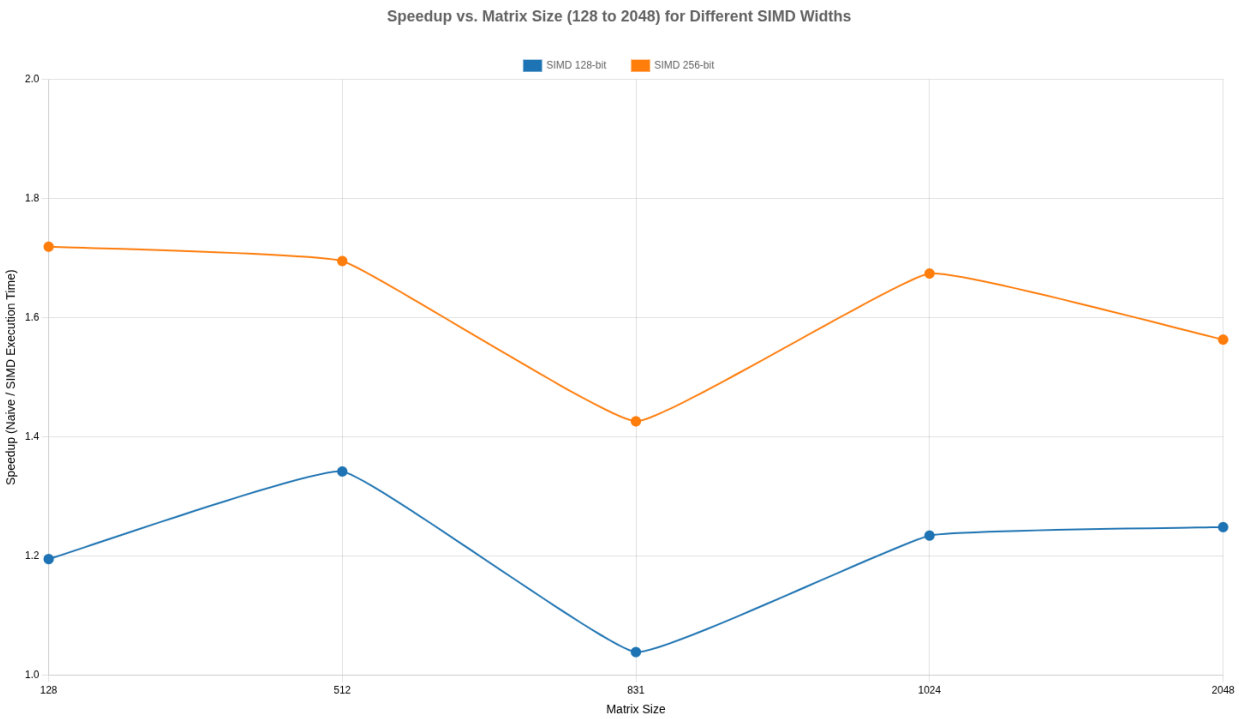
  $Speedup(256 - bit) = \frac{913\ ms}{210\ ms} = 4.35$

4. *Multi-Size Evaluation:*

- Run your implementations on matrices of various sizes.
- Analyze how performance and speedup vary with **matrix size** and **SIMD register width**.

5. *Visualization:*

- Create plots showing the **speedup** for different matrix sizes and SIMD widths.



- Choose matrix sizes that allow you to **clearly observe trends and draw meaningful conclusions**.

Answer the following:

1. Report the change in the number of instructions you observed when moving from the naive to the SIMD implementation. Justify your observations.

This table shows the number of instructions for different matrix sizes and different register sizes.

Matrix Size	Naive Multiplication	SIMD(128-bit)	% Decrease	SIMD(256-bit)	% Decrease
32	8555302	7940298	7.19%	7713632	9.84%
64	19808182	15359616	22.46%	13526777	31.71%
128	107502393	71345362	33.63%	56384797	47.55%
256	796351390	504567380	36.64%	383591739	51.83%
512	6252087801	3912573374	37.42%	2941527566	52.95%
831	26601137236	16592691830	37.62%	12480081690	53.08%
1024	49739199840	31000233607	37.67%	23193410760	53.37%
2048	397178352411	246963463231	37.82%	184444445433	53.56%

The results make sense because SIMD can do many calculations at once. With **SSE (128-bit)**, the CPU works on **2 numbers at a time**, while with **AVX (256-bit)** it works on **4 numbers at a time**. This means fewer loop steps and fewer instructions compared to the normal (scalar) version.

Since AVX can handle twice as many numbers as SSE, it cuts the instruction count even more. That's why for large matrices, **SSE gives about 37–38% fewer instructions**, while **AVX gives about 52–54% fewer instructions**.

2. Did you achieve any speedup? If so, how much, and what contributed to it? If not, what were the reasons?

   Yes,

   Contributing Factors:

   - Vectorization: SSE (2 doubles) and AVX (4 doubles) parallelize multiplications/adds, reducing computation time.
   - Small N: Compute-bound (MPKI 22-44 at N=128-256), maximizing SIMD benefits where cache misses are low.

3. Which SIMD intrinsics did you use? Justify your choice of functions.

We used the following intrinsics in the SSE (128-bit) and AVX (256-bit) implementations:

- **_mm_setzero_pd / _mm256_setzero_pd**: Initialize accumulator vector.
- **_mm_loadu_pd / _mm256_loadu_pd**: Load contiguous elements from A.
- **_mm_set_pd / _mm256_set_pd**: Pack non-contiguous elements from B.

- **_mm_add_pd / _mm256_add_pd**: multiply and add (c_vec = c_vec + (a_vec * b_vec).
- **_mm_storeu_pd / _mm256_storeu_pd**: Store vector to temporary array for reduction.

Task 1D: Rancho's Final Year Project

In this part of the assignment, you are expected to demonstrate how combining multiple optimization techniques, such as tiling, SIMD vectorization, loop unrolling, and cache-aware programming, can lead to greater performance improvements than applying each technique in isolation.

Final performance summary for Matrix Multiplication:

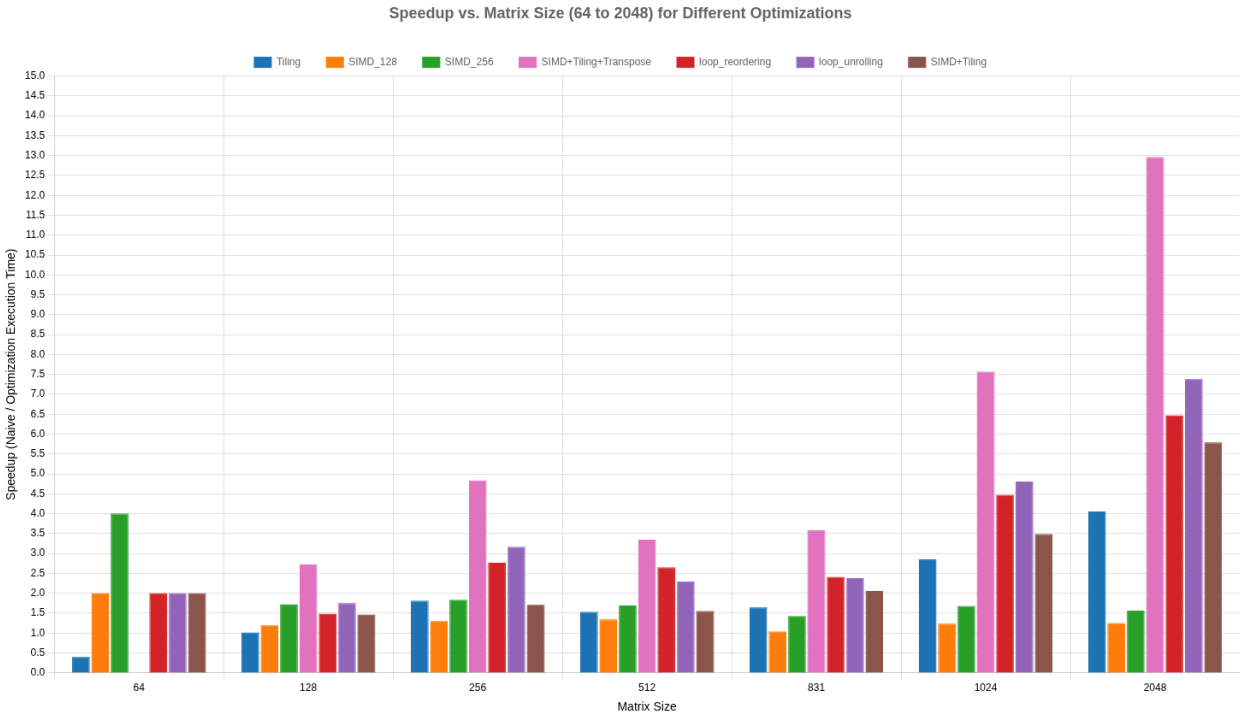
Implement a version of matrix multiplication that combines two or more of the previously explored optimization techniques.

Demonstrate how these techniques complement each other to provide synergistic performance gains, i.e., the combined benefit is greater than the sum of individual optimizations.

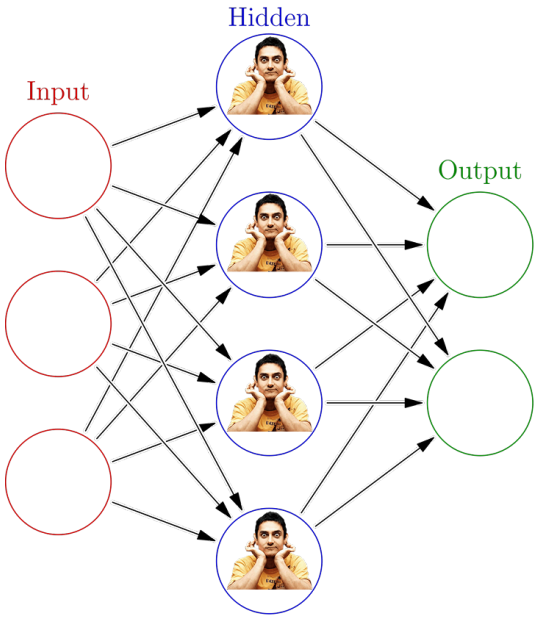
Include a plot that compares the best performance achieved by each optimization technique —

- Loop unrolling
- Loop reordering
- Tiling
- SIMD
- Tiling + SIMD

Plot this comparison against varying matrix sizes.


Speedup vs. Matrix Size (64 to 2048) for Different Optimizations
Tiling
SIMD_128
SIMD_256
SIMD+Tiling+Transpose
loop_reordering
loop_unrolling
SIMD+Tiling
Speedup (Naive / Optimization Execution Time)
15.0
14.5
14.0
13.5
13.0
12.5
12.0
11.5
11.0
10.5
10.0
9.5
9.0
8.5
8.0
7.5
7.0
6.5
6.0
5.5
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
64
128
256
512
831
1024
2048
Matrix Size

Task 1E: Confusion hi confusion hai !!



You are provided with a custom implementation of a neural network written in C++. As discussed in the background section, neural networks rely heavily on matrix multiplication for their computations.

In this task, you are required to optimize the performance of the neural network by applying the matrix multiplication techniques you have implemented earlier (e.g., tiling, SIMD, etc.). You are allowed to modify only the matrix multiplication and transpose functions.

Your goal is to incorporate one or more optimization techniques and evaluate their effect on the execution time of the neural network. Clearly report the performance improvements observed, and support your findings with appropriate measurements and analysis.

=== Matrix Multiplication Performance Benchmark ===

Matrix size: 512x512

Basic (ijk): 311.00 ms
Reordered (ikj): 92.00 ms
Unrolled: 165.00 ms
Tiled/Blocked: 123.00 ms
SIMD Vectorized: 142.00 ms

Speedup over basic implementation:
Reordered (ikj): 3.38x
Unrolled: 1.88x
Tiled/Blocked: 2.53x
SIMD Vectorized: 2.19x
Matrix: Matrix A:
0.4253 0.2669 0.1135 0.1891
-0.1826 -0.2618 0.3070 -0.3538
-0.7771 0.0910 0.4398 0.4673
-0.6226 -0.4270 -0.2459 0.0442
Matrix multiplication benchmark completed.

=== Neural Network Training Performance ===
Input Matrix:
Target Matrix:

Training with different optimizations:
Total loss: 87431.9454 over 262144.0000 elements.
Basic - Time: 1105 ms - Final Loss: 0.333526
- Counter: 8
Total loss: 87431.945355 over 262144.000000 elements.

Reordered - Time: 509 ms - Final Loss: 0.333526

- Counter: 8

Total loss: 87431.945355 over 262144.000000 elements.

Unrolled - Time: 969 ms - Final Loss: 0.333526

- Counter: 8

Total loss: 87431.945355 over 262144.000000 elements.

Tiled - Time: 780 ms - Final Loss: 0.333526

- Counter: 8

Total loss: 87431.945355 over 262144.000000 elements.

Vectorized - Time: 836 ms - Final Loss: 0.333526

- Counter: 8

Github Code:
https://github.com/Utkarshshetye/PA1_Architecture